

STUDY MATERIAL – UG (Regulations __R2025__)

Department	Artificial Intelligence and Data Science	Academic Year	2026-2027
Course Code	U25CSG11	Course Name	Object Oriented Programming using Java
Year/Semester	II/III	Class (es) & Section (s)	A
Course Coordinator	Dr.Vivek Robinson	Course handling Faculty	Ms.M.REVATHY

UNIT – I

Syllabus	UNIT I: INTRODUCTION TO OBJECT-ORIENTED PROGRAMMING AND JAVA BASICS 9 + 6 Procedure-Oriented vs Object-Oriented Java Features and Platform: JVM, JRE, JDK – Java Program Structure & Memory Layout – Classes and Objects – Static and non-static members, static and non-static Methods, Constructors – Access Modifiers, Abstraction and Encapsulation – Scanner and System classes
CO (Level)	Understand
Industry Relevance / Advanced Technologies related to the CO	Essential for console-based applications, testing, and quick prototyping. JVM ensures platform independence, critical for cross-platform enterprise solutions. JDK tools are used daily in software development pipelines.

UNIT I: Introduction to Java Programming

Index

- 1. Procedure-Oriented Programming (POP) vs Object-Oriented Programming (OOP)**
- 2. Features of Java**
- 3. Java Platform Overview**
 - **Java Virtual Machine (JVM)**
 - **Java Runtime Environment (JRE)**

- **Java Development Kit (JDK)**

4. Java Program Structure

5. Java Memory Layout

- **Stack Memory**
- **Heap Memory**
- **Method Area**
- **String Constant Pool**

6. Classes and Objects

7. Static and Non-Static Members

8. Static and Non-Static Methods

9. Constructors

- **Default Constructor**
- **Parameterized Constructor**
- **Copy Constructor**

10. Access Modifiers

- **public**
- **private**

- **protected**
- **default**

11. Abstraction

12. Encapsulation

13. Scanner Class

14. System Class

OBJECT ORIENTED PROGRAMMING USING JAVA

UNIT I - Chapter 1: Procedure-Oriented Programming (POP)

1.1 Introduction

Programming is the process of writing instructions that enable a computer to perform a specific task. Procedure-Oriented Programming (POP) divides a program into small functions or procedures, each responsible for a specific task.

1.2 Definition

Procedure-Oriented Programming (POP) is a programming paradigm in which a program is divided into procedures (functions). The emphasis is on how a task is performed rather than on the data.

Examples: C, Pascal, FORTRAN, BASIC, COBOL.

1.3 Characteristics

- Program divided into functions

Internal Quality Assurance Cell (IQAC)

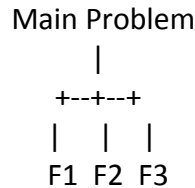
KPR Institute of Engineering and Technology

- Sequential execution
- Data shared among functions
- Focus on algorithms
- Suitable for small applications

1.4 Top-Down Approach

A large problem is divided into smaller sub-problems until each becomes easy to solve.

Diagram:



1.5 Features of POP

- Program divided into functions
- Uses Top-Down design
- Data shared using global variables or parameters
- Independent functions
- Less data security
- Suitable for small programs

1.6 Advantages

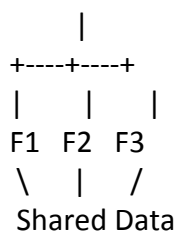
- Easy to understand
- Simple structure
- Easy debugging
- Faster execution
- Code reusability
- Less memory requirement

1.7 Disadvantages

- Less secure
- Difficult for large software
- No data hiding
- Hard maintenance
- No encapsulation, inheritance, abstraction or polymorphism

1.9 POP Architecture

PROGRAM



Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

1.11 Applications

Calculator, Payroll System, Inventory Management, Student Result Processing, Mathematical Computations.

1.12 Limitations

Not suitable for large projects, difficult maintenance, less security, poor code reuse in large applications.

Differentiate OOP from POP.

2.1 Introduction

Object-Oriented Programming (OOP) is a programming paradigm that organizes programs using classes and objects. It is suitable for developing large, secure, reusable, and maintainable software.

2.2 Definition

Object-Oriented Programming (OOP) is a programming approach in which programs are designed using classes and objects. It combines data and methods into a single unit called an object.

2.3 Need for OOP

- Better security
- Code reusability
- Easy maintenance
- Reduced complexity
- Modular programming

2.4 Characteristics

- Program organized into objects
- Data and methods combined
- Objects communicate through methods
- Supports reusability
- Provides data security
- Modular design

2.5 Features of OOP

Class

Object

Encapsulation

Abstraction

Inheritance

Polymorphism

Dynamic Binding

Message Passing

2.6 Class

A class is a blueprint used to create objects.

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

Example:

```
class Student {  
    String name;  
    void display(){}  
}
```

2.7 Object

An object is an instance of a class.

Syntax:

```
Student s = new Student();
```

2.8 Encapsulation

Encapsulation combines data and methods into one unit and protects data using private members.

Advantages: Data hiding, security, maintainability.

2.9 Abstraction

Abstraction hides implementation details and exposes only essential functionality.

2.10 Inheritance

Inheritance allows one class to acquire properties and methods of another class.

Example:

```
class Animal{}  
class Dog extends Animal{}
```

2.11 Polymorphism

Polymorphism means one interface, many forms. Example: Method overloading.

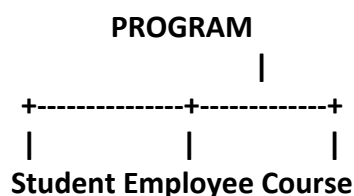
2.12 Advantages

- Better security
- Code reusability
- Easy maintenance
- Reduced code duplication
- Suitable for large applications

2.16 Real-Life Analogy

College Management System: Student, Faculty, Course, Department are objects with data and methods.

2.17 OOP Architecture



(Data + Methods)

2.19 Applications

Banking, Hospital Management, E-Commerce, Mobile Apps, Games, Desktop Applications.

POP Vs OOP

Procedure-Oriented Programming (POP) focuses on functions and procedures, whereas Object-Oriented Programming (OOP) focuses on objects that combine data and methods. OOP overcomes many limitations of POP and is widely used for developing large-scale software.

Comparison between POP and OOP

S.No	POP	OOP
1	Focuses on functions	Focuses on objects
2	Top-down approach	Bottom-up approach
3	Data and functions are separate	Data and methods are combined
4	Less secure	More secure through encapsulation
5	Uses global data	Data hiding supported
6	Suitable for small programs	Suitable for large applications
7	Less reusable	Highly reusable using inheritance
8	No inheritance	Supports inheritance
9	No polymorphism	Supports polymorphism
10	No abstraction	Supports abstraction
11	Maintenance is difficult	Maintenance is easier
12	Code duplication is common	Reduces code duplication
13	Real-world modelling is difficult	Supports real-world modelling
14	Examples: C, Pascal	Examples: Java, C++, Python, C#
15	Security is low	Security is high

3 Java Features

1. Simple

Java has a clean syntax, automatic memory management and removes complex features such as pointers (direct pointer arithmetic).

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

2. Object-Oriented

Java is based on classes and objects and supports encapsulation, abstraction, inheritance and polymorphism.

3. Platform Independent

Java programs are compiled into bytecode, which runs on any operating system with a JVM. This is known as 'Write Once, Run Anywhere (WORA)'.

4. Portable

Java programs can be moved from one platform to another without modification.

5. Robust

Java provides exception handling, automatic garbage collection and strong type checking.

6. Secure

Java provides bytecode verification, access control and runtime security to protect applications.

7. Architecture Neutral

Compiled bytecode is independent of processor architecture.

8. Multithreaded

Java allows multiple threads to execute simultaneously, improving application performance.

9. Distributed

Java supports distributed computing using networking APIs such as sockets and RMI.

10. Dynamic

Java loads classes dynamically during runtime when required.

11. High Performance

The Just-In-Time (JIT) compiler converts frequently used bytecode into native machine code for faster execution.

12. Automatic Garbage Collection

Unused objects are automatically removed from memory by the Garbage Collector.

Applications of Java

- Desktop applications
- Web applications
- Enterprise software
- Android applications
- Cloud applications
- Banking systems
- Scientific applications

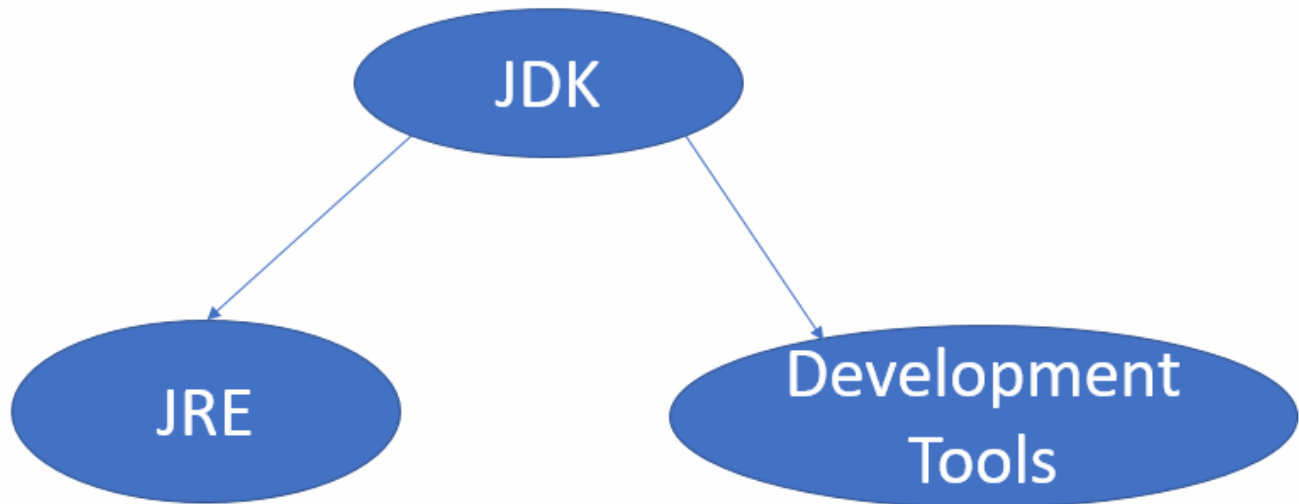
4 JVM, JRE, JDK

4.1 What is JDK ?

Java Development Kit (JDK) is software development environment which is used to develop java applications and applets.

JDK help us to develop Java application and have all below resources to complete or create a java application:

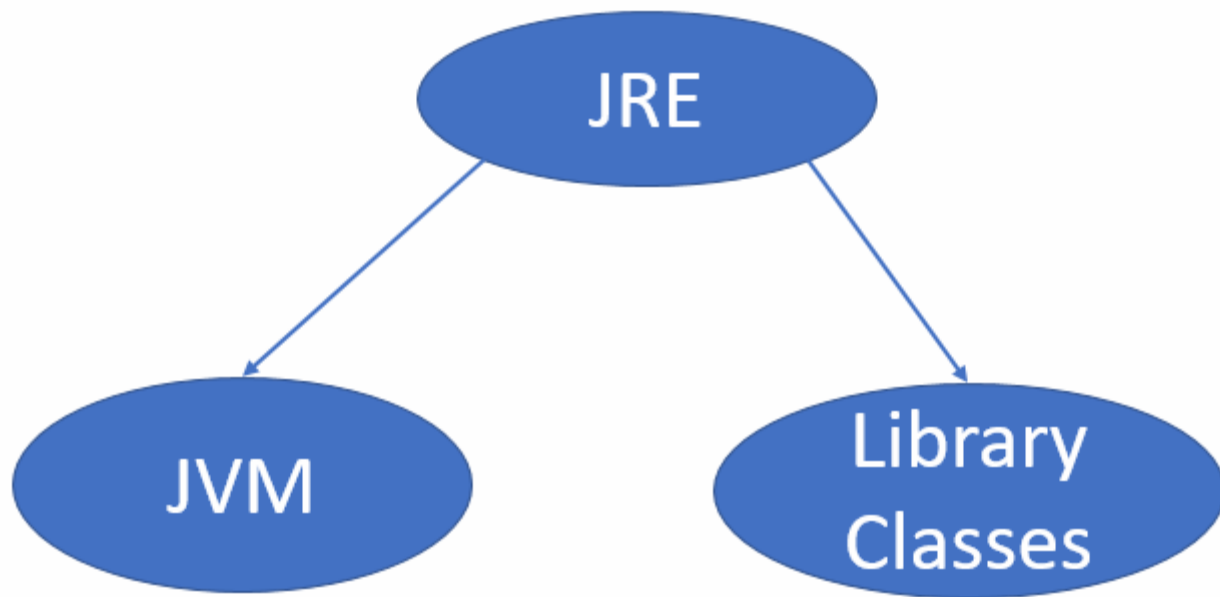
java runtime environment (JVM), Interpreter/loader (Java), compiler (javac), archiver(jar) and java documentation (Java doc).



4.2 What is JRE?

JRE stands for “Java Runtime environment”. JRE is integral part of the Java Development Kit (JDK).

It Provides us an environment and helps us to execute or run Java program. JRE physically in the form of files on your system.

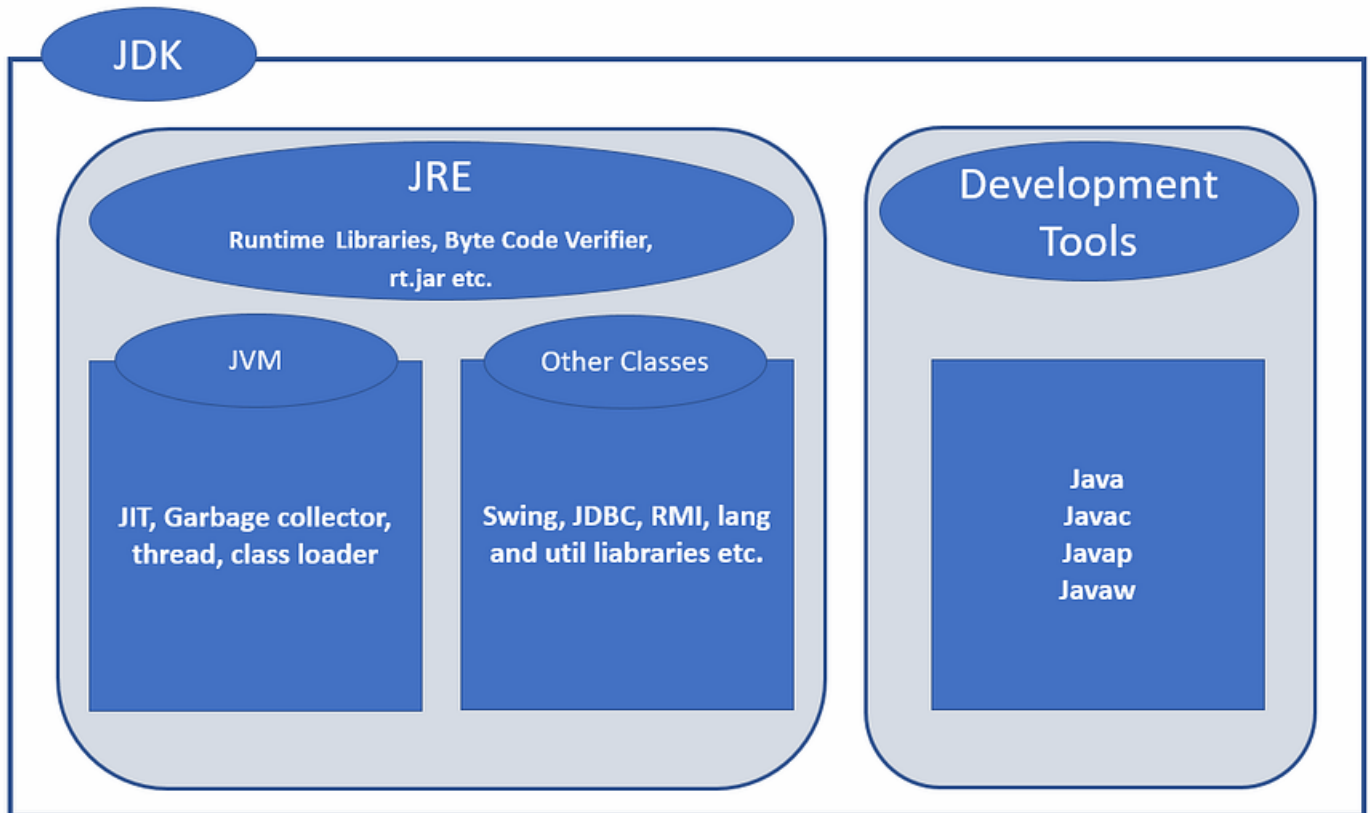


4.3 What is JVM?

JVM stands for Java Virtual Machine. Both JDK and JRE contains JVM.

Java virtual machine (JVM) is a kind of unique virtual machine that enables a computer to run Java programs as well as programs written in other languages that are also compiled to Java bytecode.

JVM is completely responsible for the execution of running Java programs and manage program memory using garbage collector. JVM provides us the capability to run Java program on any operating system.



4.4 What is JIT?

JIT stands for Just-In-Time compiler. It is an essential part of JRE.

It is mainly responsible for performance optimization of Java-based applications at run time or execution time.

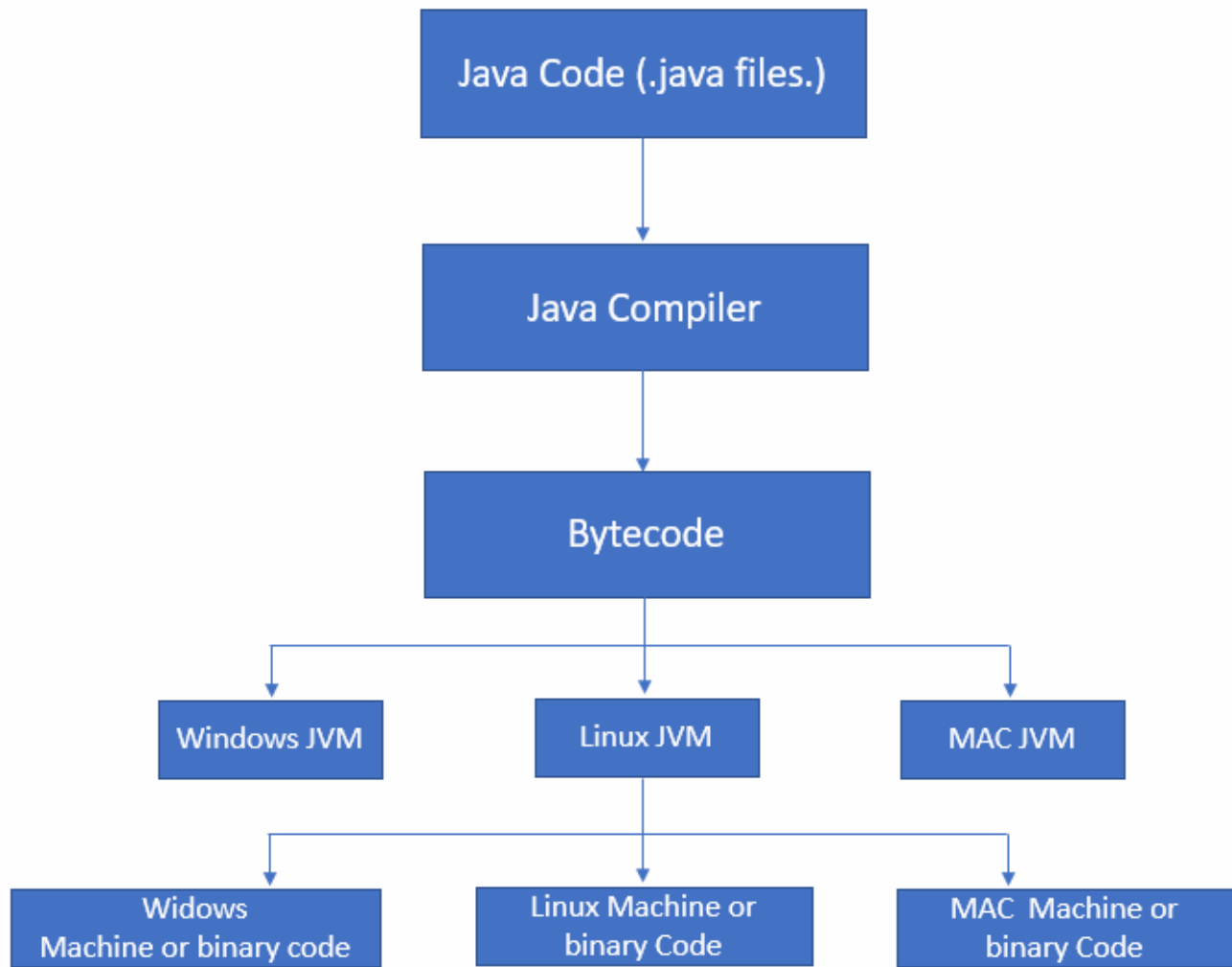
4.5 What is ByteCode?

Java also called as platform independent because of Bytecode.

The Bytecode gets generate in the form of .class file as soon as the Java program gets compiled. Bytecode is an instruction for JVM to covert it in to machine language. The Bytecode makes java to write once, run anywhere (WORA).

Java is platform independent. Because the Java compiler converts the source code to bytecode, which is Intermediate Language called as bytecode which can be executed on any platform or operating system.

Bytecode alone is of no use and non-runnable as it requires an interpreter (JVM) to run on any machine.



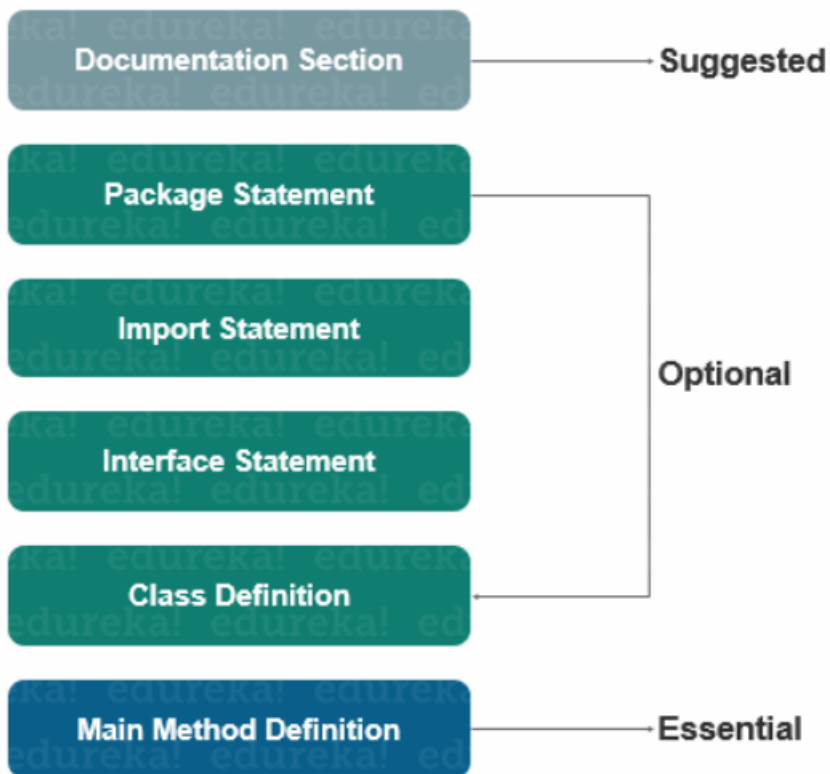
Java programming language is platform-independent and a secure programming language. With various applications, Java programming language has been in demand for the last two decades. The out-of-the-box features help java stand apart.

5 Structure of Java Program & memory management:

5.1 Documentation Section

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology



It is used to improve the readability of the program. It consists of comments in Java which include basic information such as the method's usage or functionality to make it easier for the programmer to understand it while reviewing or debugging the code. A Java comment is not necessarily limited to a confined space, it can appear anywhere in the code.

The compiler ignores these comments during the time of execution and is solely meant for improving the readability of the Java program.

There are three types of comments that Java supports

- Single line Comment
- Multi-line Comment
- Documentation Comment

Let's take a look at an example to understand how we can use the above-mentioned comments in a Java program.

```
1// a single line comment is declared like this
2/* a multi-line comment is declared like this
3and can have multiple lines as a comment */
4/** a documentation comment starts with a delimiter and ends with */
```

5.2 Package Statement

A provision in Java allows you to declare your classes in a collection called package. There can be only one package statement in a Java program, which must be at the beginning of the code before any class or interface declaration. This statement is optional, for example, take a look at the statement below.

```
1package student;
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

This statement declares that all the classes and interfaces defined in this source file are a part of the student package. And only one package can be declared in the source file.

5.3 Import Statement:

Many predefined classes are stored in packages in Java, an import statement is used to refer to the classes stored in other packages. An import statement is always written after the package statement but it has to be before any class declaration.

We can import a specific class or classes in an import statement. Take a look at the example to understand how import statement works in Java.

```
1import java.util.Date; //imports the date class
2import java.applet.*; //imports all the classes from the java applet package
```

5.4 Interface Section

This section is used to specify an interface in Java. It is an **optional** section which is mainly used to implement multiple inheritance in Java. An interface is a lot similar to a class in Java but it contains only constants and method declarations.

An interface cannot be instantiated but it can be implemented by classes or extended by other interfaces.

```
1interface stack{
2void push(int item);
3void pop();
4}
```

5.5 Class Definition

A Java program may contain several class definitions, classes are an essential part of any Java program. It defines the information about the user-defined classes in a program.

A class is a collection of variables and methods that operate on the fields. Every program in Java will have at least one class with the main method.

5.6 Main Method Class

The main method is from where the execution actually starts and follows the order specified for the following statements. Let's take a look at a sample program to understand how it is structured.

```
1public class Example{
2//main method declaration
3public static void main(String[] args){
4System.out.println("hello world");
5}
6}
```

Let's analyze the above program line by line to understand how it works.

public class Example This creates a class called Example. You should make sure that the class name starts with a capital letter, and the public word means it is accessible from any other classes.

5.7 Comments

To improve the readability, we can use comments to define a specific note or functionality of methods, etc for the programmer.

5.8 Braces

The curly brackets are used to group all the commands together. To make sure that the commands belong to a class or a method.

public static void main

- When the main method is declared public, it can also be used outside of this class.
- The word static means that we want to access a method without making its objects. As we call the main method without creating any objects.
- The word void indicates that it does not return any value. The main is declared as void because it does not return any value.
- Main is the method, which is an essential part of any Java program.
-

5.9 String[] args

It is an array where each element is a string, which is named as args. You can pass the input parameter if you run the Java code through a console. The main() takes it as an input.

```
System.out.println();
```

The statement is used to print the output on the screen where the system is a predefined class, out is an object of the PrintWriter class. The method println prints the text on the screen with a new line. All Java statements end with a semicolon.

Introduction.

Memory Management in Java

Memory Management in Java refers to allocating and deallocating memory to Java objects in areas called Stack and Heap. Java has an automatic memory deallocation system known as Garbage Collector. This article presents some concepts of Java Memory management and the working of a Garbage Collector in JVM.

What is Memory Management In Java?

Memory management in Java is the process of allocating working memory space to new objects and properly removing unreferenced objects to create space for those new object allocations. Generally, Java developers don't have to deal with memory handling in Java directly because the Garbage Collector has the duty of memory deallocation handling and it runs automatically. But many times due to improper resource handling from code the memory consumed by the Java applications is way higher than what Garbage Collections cleans up and as a result, there occurs memory leakage and then there occurs the need for performance tuning for proper memory management.

Types of Java Memory Management

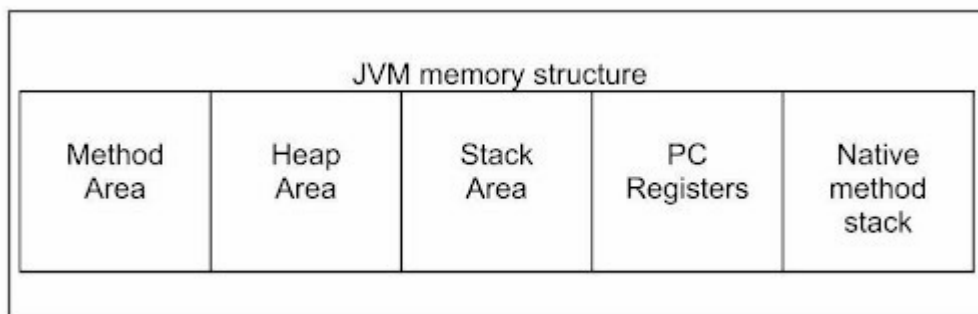
There are two major concepts for Java Memory Management

- JVM Memory structure
- Working of Garbage Collector.

1) JVM Memory Structure

The Java memory model describes how threads in the Java programming language interact through memory. JVM memory structure has different areas where the objects of the programs reside.

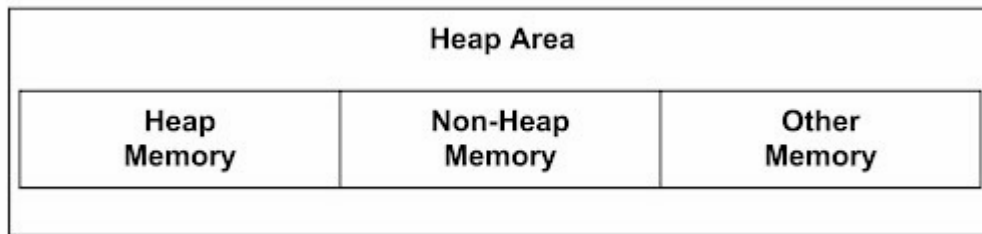
JVM Memory Is Divided into multiple parts:



Method Area

The method area is a logical part of the heap area that is created at the time of Virtual Machine startup. The method area is allocated for method data, class structures, constructor field data as well as for interfaces and special methods in a class. Depending on system configuration, the Method area can be fixed or dynamic required by the computation. It need not be contiguous. Garbage collection is mandatory in the heap, but the Method area may or may not be garbage collected though it is a logical part of the heap area.

Heap Area



The heap area is a shared runtime data area that can be allocated to real objects of all class instances and arrays. Depending on the configuration of the system, the size of the heap can be dynamic or fixed. It is initiated during the Java Virtual Machine Startup. Heap Memory allocated to the objects is reclaimed by the Garbage Collector. By default, the maximum size of the heap is 64 Mb. When you use a new keyword, an object is allocated memory in the heap and its memory is referenced using a variable from the thread's stack.

There is always only one Heap for a running Java Virtual Machine process. When the heap is filled the garbage collector clears the objects.

VM options to configure heap size :

To set the maximum Java heap size: -Xmx

To set the initial Java heap size: -Xms

JVM Stack area

A Stack is created simultaneously with the creation of a thread. Stack is used to store method-specific values that are short-lived and are referenced to other objects in the heap. The stack memory need not be contiguous. Its size can either be fixed or dynamic and can be chosen independently when it is created.

What is a Stack Frame?

The stack frame is the data structure of the thread's data. The thread data represents the state of the thread in the current method.

- Stack frame stores partial results, and data also performs dynamic linking.
- Each frame contains its own Local Variable Array (LVA), Frame Data (FD), and Operand Stack (OS).
- The sizes of LVA, FD and OS are determined at the compile time
- In a thread of control, only one frame is active at a time i.e. current frame. This frame is called the current frame and the class of method is called the current class.
- In case the method invokes another method or the method is completed, the frame stops the current method.
- The frame can only be referenced by the thread that created it.

Program Counter (PC) Registers

Every JVM thread that is created has an associated PC register. The value of the program counter is undefined in the native method while the non-native method has a Program Counter that stores the address of the JVM instructions currently being executed. It also stores the native pointer or a return address.

Native Method Stack

Native Method stack is also known as C stack. Native method stacks are written in other programming languages, not Java. The Java Native Interface (JNI) calls the native stack. This type of memory is allocated to the thread when it is created. Similar to other types it can be fixed or dynamic. Native stack performance depends on the Operating System. <https://www.youtube.com/watch?v=vz6vSZRuS2M>

2) Working as a Garbage Collector

When a program runs in Java, objects are allocated memory in the heap. Garbage Collector tries to keep as much free working memory space as possible. Heap is the only part of the Java memory where Garbage Collection is possible.

What is the Java Garbage Collection?

Garbage Collection is the process of collecting memory occupied by unreferenced objects. Garbage Collector is controlled by Java Virtual Machine (JVM), JVM chooses when to run the garbage collector. Garbage Collection is done whenever JVM detects the low availability of working memory resources. JVM also considers our request to perform garbage collection. But it's not always guaranteed that JVM will comply with our request.

Garbage Collector Types

JVM chooses from the 3 types of Garbage Collectors based on the underlying hardware and does the deallocation. Programmers can also choose which garbage collector should be used.

The 3 types of garbage collectors are

- Serial GC: It is a single-threaded collector, mostly used in small applications that use small amounts of data. Specify the given command line to enable serial GC: -XX:+UseSerialGC
- Parallel GC: Also known as throughput collector. It uses multiple threads for the garbage collection process. You can enable parallel GC simply by specifying the given command line: -XX:+UseParallelGC
- Mostly Concurrent GC: This type of Garbage Collector runs Concurrent to the application. However, it's not 100% Concurrent but the pause time of the threads is kept as low as possible to optimize the GC Performance. <https://www.youtube.com/watch?v=XXOaCV5xm9s>

8-Classes and Objects

Introduction

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

A class is a blueprint/ template for creating objects. An object is an instance of a class that occupies memory at runtime. Java programs are built using classes and objects.

Class

Definition: A class is a user-defined data type that contains data members, methods, constructors, static members and nested classes.

Syntax:

```
class Student {  
    String name;  
    int age;  
    void display(){}  
}
```

Components of a Class

- Instance variables
- Methods
- Constructors
- Static members
- Nested classes
- Initialization blocks

Class Naming Rules

- Begin with a letter, _ or \$
- Cannot start with a digit
- Avoid Java keywords
- Use PascalCase naming convention
- Public class name should match the file name

Object

Definition: An object is an instance of a class that contains state (data) and behaviour (methods).

Syntax:

```
Student s = new Student();
```

Object Creation

Step 1: Declare reference variable

Step 2: Create object using new

Step 3: Access members using (.) operator

Object Creation Rules

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

- Object must be created from a class
- Use the new keyword
- Multiple objects can be created
- Cannot create an object of an abstract class or interface
- Cannot access instance members using the class name

Reference Variable

A reference variable stores the address of an object.

Example:

```
Student s = new Student();
```

Memory Representation

STACK	HEAP
+-----+	+-----+
s --- ----->	Student Object
+-----+	+-----+

Accessing Members

Use the dot (.) operator.

Example:

```
s.name="Revathy";
```

```
s.display();
```

Example Program

```
class Student {  
    String name="Revathy";  
    void display(){  
        System.out.println(name);  
    }  
    public static void main(String[] args){  
        Student s=new Student();  
        s.display();  
    }  
}
```

Output:

Revathy

Advantages

- Real-world modelling
- Code reusability

- Easy maintenance
- Modular programming

9- Static and Non Static members and Methods

Java is one of the most popular object-oriented programming languages used for developing web applications, desktop software, Android apps, and enterprise systems. One of the most important concepts in Java is the difference between static and non-static members. Understanding this concept helps beginners write efficient and organized Java programs. In Java, variables, methods, blocks, and classes can be declared as static or non-static depending on how they should behave in memory.

Static members belong to the class itself, while non-static members belong to objects of the class. This difference affects memory allocation, accessibility, and performance. Beginners often get confused about when to use static and when to use non-static elements in Java programs. Learning this topic is essential for Java interviews, coding tests, and real-world application development. In this guide, we will explain static and non-static concepts in simple language with examples and practical use cases.

What Does Static Mean in Java?

The keyword “static” in Java is used to create members that belong to the class instead of an object. This means you do not need to create an object of the class to access static variables or methods. Static members are shared among all objects of the class and are stored only once in memory.

Static is commonly used for utility methods, constants, counters, and the main method in Java. Since static members are shared, they help save memory and improve performance. For example, if a college has one common college name for all students, that variable can be declared static because it remains the same for every object.

Static methods can directly access static variables but cannot directly access non-static variables because non-static members require object creation. The static keyword is widely used in Java programming for better memory management and code optimization. Understanding **static helps developers create cleaner and** more efficient applications.

What Does Non-Static Mean in Java?

Non-static members in Java belong to objects instead of the class itself. These members are also known as instance members because every object gets its own separate copy. To access non-static variables or methods, you must first create an object of the class.

Non-static members are useful when data changes from one object to another. For example, student names, roll numbers, and marks should be non-static because each student has unique values. Every time a new object is created, separate memory is allocated for non-static members.

Non-static methods can access both static and non-static members directly because they work with object instances. These methods are mainly used for performing object-specific operations. In Java, most **real-world** programs rely heavily on non-static concepts because objects represent real entities. Understanding non-static members is essential for mastering object-oriented programming principles like encapsulation and inheritance.

Key Difference Between Static and Non-Static in Java

The primary difference between static and non-static in Java is ownership. Static members belong to the class, while non-static members belong to objects. Static members are shared among all objects, whereas non-static members have separate copies for each object.

Another major difference is memory allocation. Static members are loaded only once when the class is loaded into memory. Non-static members are loaded every time a new object is created. This means static members consume less memory compared to non-static members.

Static methods can be called directly using the class name, while non-static methods require object creation. Static members are mainly used for common data shared by all objects, whereas non-static members are used for object-specific data.

In terms of execution, the JVM loads static members before object creation. Non-static members become active only after objects are instantiated. Knowing these differences helps programmers decide which type of member should be used in different situations for better application design and performance.

Static vs Non-Static Members in Java

In Java, members of a class include variables, methods, blocks, and nested classes. These members can either be static or non-static depending on their purpose. Static members are associated with the class itself, while non-static members are associated with individual objects.

Static variables store common data shared among all objects. Static methods perform operations related to the class rather than object data. Static blocks are used for initializing static variables. On the other hand, non-static variables store unique object data, and non-static methods work on instance values.

One major advantage of static members is memory efficiency because only one copy exists. However, non-static members provide flexibility since each object can have different values. Developers should carefully choose between static and non-static members based on application requirements.

For example, in a banking application, the bank name can be static because it is common for all customers. However, account balance should be non-static because every customer has a unique balance. Understanding these differences improves coding practices and application performance.

Static Variables

A static variable in Java is a class-level variable shared among all objects of the class. It is declared using the static keyword and stored in a common memory area. Since only one copy exists, changes made by one object affect all other objects.

Static variables are commonly used for constants, counters, configuration settings, and shared data. For example, if all employees work for the same company, the company name can be declared static because it remains the same for every employee object.

One important feature of static variables is that they can be accessed directly using the class name without creating an object. This improves memory efficiency and simplifies code structure. Static variables are initialized only once when the class loads into memory.

However, overusing static variables can reduce flexibility because all objects share the same value. Therefore, developers should use static variables only when data needs to remain common across all instances of a class.

Non-Static Variables

Non-static variables are instance variables that belong to individual objects. Every time a new object is created, a separate copy of non-static variables is allocated in memory. These variables store unique information for each object.

For example, in a student management system, each student has a different name, roll number, and marks. These values should be non-static because they vary from object to object. Changes made in one object do not affect other objects.

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

Non-static variables are created when an object is instantiated and destroyed when the object is removed from memory. They are essential for representing real-world entities in object-oriented programming.

Unlike static variables, non-static variables cannot be accessed directly using the class name. An object reference is required to access them. These variables provide flexibility and support dynamic behavior in Java applications.

Using non-static variables correctly helps maintain data integrity and object independence. Most real-world Java programs rely heavily on non-static variables because applications usually deal with unique object data rather than shared information.

Static Methods in Java with Example

A static method in Java belongs to the class rather than an object. It can be accessed directly using the class name without creating an object. Static methods are commonly used for utility functions and operations that do not depend on object data.

One important rule is that static methods can directly access only static members. They cannot directly access non-static members because non-static members require object creation. The most common example of a static method is the `main()` method in Java.

Example:

```
class Demo {  
    static void display() {  
        System.out.println("Static Method");  
    }  
  
    public static void main(String[] args) {  
        Demo.display();  
    }  
}
```

In this example, the `display()` method is called using the class name without object creation. Static methods improve performance and memory usage because they are loaded only once. They are widely used in utility classes like `Math` and `Collections` in Java.

Non-Static Methods in Java with Example

Non-static methods are instance methods that belong to objects. To call these methods, you must create an object of the class. These methods can directly access both static and non-static variables because they work with object instances.

Non-static methods are mainly used for operations that depend on object-specific data. For example, a student object may have methods to display marks, calculate grades, or update information.

Example:

```
class Student {  
    void show() {  
        System.out.println("Non-Static Method");  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.show();  
    }  
}
```

```
}
```

In this example, the show() method is called using an object reference. Non-static methods provide flexibility and support dynamic behavior in Java applications.

Most object-oriented programs use non-static methods because they help represent real-world entities and operations. Understanding instance methods is essential for creating reusable and scalable Java applications.

Static Block in Java

A static block in Java is used for initializing static variables. It executes automatically when the class is loaded into memory before the main() method runs. Static blocks are mainly used when complex initialization is required.

A class can contain multiple static blocks, and they execute in the order they appear. Static blocks run only once during the class-loading process, regardless of how many objects are created.

Example:

```
class Test {  
    static {  
        System.out.println("Static Block Executed");  
    }  
  
    public static void main(String[] args) {  
        System.out.println("Main Method");  
    }  
}
```

In this example, the static block executes before the main method. Static blocks are useful for loading configuration files, database drivers, or initializing static resources.

Because they run only once, static blocks improve efficiency for tasks that should not repeat multiple times. However, developers should avoid placing unnecessary logic inside static blocks to maintain code readability and maintainability.

Non-Static Block in Java

A non-static block in Java is also called an instance initialization block. It executes every time an object of the class is created. These blocks are mainly used for initializing instance variables before constructors execute.

Unlike static blocks, non-static blocks run once for each object creation. They are useful when multiple constructors share common initialization code. This reduces code duplication and improves maintainability.

Example:

```
class Example {  
    {  
        System.out.println("Non-Static Block");  
    }  
  
    Example() {  
        System.out.println("Constructor");  
    }  
  
    public static void main(String[] args) {  
        new Example();  
    }  
}
```

```
}
```

In this example, the non-static block executes before the constructor. Non-static blocks are less commonly used compared to constructors, but they still play an important role in object initialization.

Understanding instance blocks helps developers learn the sequence of execution in Java classes and improves their understanding of object creation processes.

Static Class in Java

Java does not allow top-level classes to be declared static, but nested classes can be static. A static nested class belongs to the outer class rather than its objects. This means it can be accessed without creating an object of the outer class.

Static nested classes are mainly used for logically grouping related classes together. They improve code organization and reduce unnecessary object creation.

Example:

```
class Outer {  
    static class Inner {  
        void display() {  
            System.out.println("Static Nested Class");  
        }  
    }  
}  
  
    public static void main(String[] args) {  
        Outer.Inner obj = new Outer.Inner();  
        obj.display();  
    }  
}
```

In this example, the inner class is accessed without creating an object of the outer class. Static nested classes improve memory efficiency and make code easier to manage in large applications.

They are commonly used in frameworks and libraries where helper classes need close association with the outer class without depending on object instances.

Why Main Method is Static in Java

The main() method in Java is declared static because the Java Virtual Machine (JVM) calls it before creating any object of the class. If main() were non-static, the JVM would need to create an object first, which would complicate program execution.

Since static methods belong to the class, the JVM can directly access the main() method using the class name. This makes the program startup process simple and efficient.

The syntax of the main method is:

```
public static void main(String[] args)
```

Here, static ensures the method can run without object creation. The JVM uses this method as the entry point of every Java application.

Making main() static also improves performance because the method loads once during class loading. Understanding why the main method is static helps beginners understand the relationship between class loading, object creation, and program execution in Java.

Memory Allocation of Static and Non-Static Members

Memory allocation is one of the biggest differences between static and non-static members in Java. Static members are stored in the class memory area and loaded only once when the class is loaded. This means all objects share the same copy of static members.

Non-static members are stored in heap memory and created separately for each object. Every time a new object is created, fresh memory is allocated for instance variables and methods.

Because static members use shared memory, they help reduce memory consumption. Non-static members consume more memory because each object maintains its own copy. However, they provide flexibility and support unique object behavior.

Accessing Static and Non-Static Variables

Static variables can be accessed directly using the class name, while non-static variables require object creation. This is one of the most fundamental differences in Java programming.

Example of static variable access:

```
class Demo {  
    static int number = 10;  
  
    public static void main(String[] args) {  
        System.out.println(Demo.number);  
    }  
}
```

Example of non-static variable access:

```
class Demo {  
    int number = 20;  
  
    public static void main(String[] args) {  
        Demo obj = new Demo();  
        System.out.println(obj.number);  
    }  
}
```

Static access is faster and more memory-efficient because no object creation is needed. Non-static access provides flexibility because each object can hold different values.

Understanding how to access these variables correctly helps developers avoid common programming errors and improves code readability. Choosing the correct access method is essential for writing efficient and scalable Java applications.

Can Static Methods Access Non-Static Members?

Static methods in Java cannot directly access non-static members because non-static members belong to objects, while static methods belong to the class itself. Since a static method can execute without object creation, it does not know which object's non-static data should be used. This is one of the most common concepts asked in Java interviews and programming exams.

However, a static method can access non-static members indirectly by creating an object of the class. Once an object is created, the static method can use that object reference to access instance variables and instance methods.

Example:

```
class Demo {
```

```
int num = 50;

static void display() {
    Demo obj = new Demo();
    System.out.println(obj.num);
}

public static void main(String[] args) {
    display();
}
}
```

In this example, the static method accesses the non-static variable through an object. Understanding this rule helps programmers avoid common compile-time errors in Java applications.

Example:

```
class Example {
    static int number = 100;

    void show() {
        System.out.println(number);
    }

    public static void main(String[] args) {
        Example obj = new Example();
        obj.show();
    }
}
```

In this example, the non-static method show() directly accesses the static variable. Understanding this relationship between static and non-static members is essential for writing efficient and error-free Java programs.

Real-Life Example of Static and Non-Static in Java

Real-life examples make it easier to understand static and non-static concepts in Java. Consider a college management system. The college name remains the same for all students, while student names and roll numbers are different for each student.

In this case, the college name should be declared static because it is shared among all objects. Student names and roll numbers should be non-static because every student has unique information.

Example:

```
class Student {
    static String college = "ABC College";
    String name;

    Student(String n) {
        name = n;
    }
}
```

```
void display() {  
    System.out.println(name + " - " + college);  
}  
  
public static void main(String[] args) {  
    Student s1 = new Student("Rahul");  
    Student s2 = new Student("Aman");  
  
    s1.display();  
    s2.display();  
}  
}
```

This example clearly shows how static members store common data while non-static members store object-specific data. Real-world applications frequently use both concepts together for efficient memory management and better program design.

Advantages of Static Members in Java

Static members provide several advantages in Java programming, especially when dealing with shared data and utility operations. One major advantage is memory efficiency. Since static variables are created only once, all objects share the same copy, reducing memory usage significantly.

Another advantage is easy accessibility. Static methods and variables can be accessed directly using the class name without creating objects. This simplifies code and improves performance. Static members are also useful for utility classes such as the Math class in Java.

Static methods execute faster because they are loaded during class loading. They are ideal for helper functions, constants, counters, and configuration settings. Static blocks also help initialize resources only once during program execution.

Using non-static members correctly helps developers create scalable, reusable, and maintainable software applications.

Static vs Non-Static: Comparison Table

Feature	Static	Non-Static
Belongs To	Class	Object
Memory Allocation	Once per class	Every object
Access Method	Class name	Object reference
Object Required	No	Yes
Data Sharing	Shared among objects	Separate for each object
Memory Usage	Low	Higher
Method Access	Only static directly	Both static and non-static

Execution Time	Faster	Depends on object
Main Purpose	Common data	Object-specific data
Example	Company Name	Employee Name

10-CONSTRUCTORS

In Java, a constructor is a block of codes similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object. Every time an object is created using the new() keyword, at least one constructor is called. It calls a default constructor if there is no constructor available in the class.

In such case, Java compiler provides a default constructor by default. There are two types of constructors in Java: no-arg constructor, and parameterized constructor.

Rules for creating Java constructor

There are two rules defined for the constructor.

1. Constructor name must be the same as its class name
2. A Constructor must have no explicit return type
3. A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors

There are two types of constructors in Java:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Java Default Constructor

A constructor is called "Default Constructor" when it doesn't have any parameter.

Syntax of default constructor:

```
<class_name>(){}
```

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

//Java Program to create and call a default constructor

```
class Bike1{
//creating a default constructor
Bike1(){System.out.println("Bike is created");}
//main method
public static void main(String args[]){
//calling a default constructor
Bike1 b=new Bike1();
}
}
```

Output:

Bike is created

Q) What is the purpose of a default constructor?

The default constructor is used to provide the default values to the object like 0, null, etc., depending on the type.

Example of default constructor that displays the default values

//Let us see another example of default constructor

//which displays the default values

```
class Student3{
int id;
String name;
//method to display the value of id and name
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the object
s1.display();
s2.display();
}
}
```

Output:

0 null

0 null

Explanation: In the above class, you are not creating any constructor so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.

Java Parameterized Constructor

A constructor which has a specific number of parameters is called a parameterized constructor.

Why use the parameterized constructor?

The parameterized constructor is used to provide different values to distinct objects. However, you can provide the same values also.

Example of parameterized constructor

In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

//Java Program to demonstrate the use of the parameterized constructor.

```
class Student4{
int id;
String name;
//creating a parameterized constructor
Student4(int i,String n){
id = i;
name = n; }
}
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

```
//method to display the values
void display()
{
    System.out.println(id+" "+name);
}
public static void main(String args[]){
    //creating objects and passing values
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    //calling method to display the values of object
    s1.display();
    s2.display();
}
}
```

Output:

111 Karan

222 Aryan

Constructor Overloading in Java

In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

Example of Constructor Overloading

//Java program to overload constructors

```
class Student5{
    int id;
    String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n){
        id = i;
        name = n;
    }
    //creating three arg constructor
```

Java Copy Constructor

There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in Java. They are:

o By constructor

o By assigning the values of one object into another

o By clone() method of Object class

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

In this example, we are going to copy the values of one object into another using Java constructor.

//Java program to initialize the values from one object to another object.

```
class Student6{
int id;
String name;
//constructor to initialize integer and string
Student6(int i,String n){
id = i;
name = n;
}
//constructor to initialize another object
Student6(Student6 s){
id = s.id;
name =s.name;
}
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
Student6 s1 = new Student6(111,"Karan");
Student6 s2 = new Student6(s1);
s1.display();
s2.display();
}
}
```

Output:

111 Karan

111 Karan

Copying values without constructor

We can copy the values of one object into another by assigning the objects values to another object. In this case, there is no need to create the constructor.

```
1. class Student7{
2. int id;
3. String name;
4. Student7(int i,String n){
5. id = i;
6. name = n;
7. }
8. Student7(){ }
9. void display(){System.out.println(id+" "+name);}
11. public static void main(String args[]){
12. Student7 s1 = new Student7(111,"Karan");
13. Student7 s2 = new Student7();
14. s2.id=s1.id;
```

```
15. s2.name=s1.name;
16. s1.display();
17. s2.display();
18. }
19. }
```

Test it Now

Output:

```
111 Karan
111 Karan
```

Java Copy Constructor

There is no copy constructor in Java. However, we can copy the values from one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in Java.

They are:

- o By constructor
- o By assigning the values of one object into another
- o By clone() method of Object class

In this example, we are going to copy the values of one object into another using Java constructor.

//Java program to initialize the values from one object to another object.

```
class Student6{
int id;
String name;
//constructor to initialize integer and string
Student6(int i,String n){
id = i;
name = n;
}
//constructor to initialize another object
Student6(Student6 s){
id = s.id;
name =s.name;
}
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
Student6 s1 = new Student6(111,"Karan");
Student6 s2 = new Student6(s1);
s1.display();
s2.display();
}
}
```

Output:

```
111 Karan
```

111 Karan

Copying values without constructor

We can copy the values of one object into another by assigning the objects values to another object. In this case, there is no need to create the constructor.

```
class Student7{
int id;
String name;
Student7(int i,String n){
id = i;
name = n;
}
Student7(){
}
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
Student7 s1 = new Student7(111,"Karan");
Student7 s2 = new Student7();
s2.id=s1.id;
s2.name=s1.name;
s1.display();
s2.display();
}
}
```

Output:

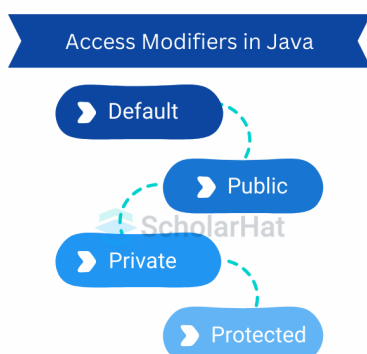
111 Karan

111 Karan

link: <https://www.youtube.com/watch?v=pgBk8HC7jbU&t=52s>

11. Access Modifiers in Java: Default, Private, Public, Protected

Access modifiers in Java decide who can access classes, methods, and variables in your code. They play a key role in organizing your program, protecting data, and controlling visibility across different parts of your application. By using access modifiers wisely, you embrace the principle of encapsulation—a core concept of object-oriented programming that leads to more secure, maintainable, and scalable code.



What is an Access Modifier in Java?

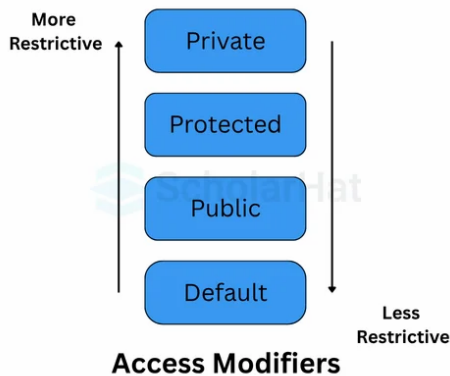
Access Modifier in Java are keywords that define the visibility or accessibility of classes, methods, variables, and constructors within a program. They guide the Java compiler on where a particular element can be accessed from, ensuring better control and encapsulation in your code. Java provides four main access modifiers:

- **public:** The element is accessible from any class, regardless of the package.
- **private:** The element is accessible only within the same class.
- **protected:** The element is accessible within the same package and in subclasses, even in different packages.
- **default (no modifier):** Accessible only within the same package (also called package-private).

As a real-time example, we can choose Facebook, where users can control their posts.

There are three types of Access Modifiers: "Public Access Specifier," "Protected Access Specifier," and "Private Access Specifier."

- If anyone wants to make the status visible to the public, they can choose "Public Access Specifiers."
- If anyone wants to make their status visible to only their friends, it is called a "Protected Access Specifier."
- Lastly, if someone wants to make the status visible only to themselves, they can use the "Private Access Specifier."



Types of Access Modifiers in Java;

There are four types of Access Modifier in Java:

1. Default Access Modifier
2. Private Access Modifier
3. Protected Access Modifier
4. Public Access Modifier

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

1. Default Access Modifier in Java

In Java, the default access modifier is automatically applied when no other access modifier (public, private, or protected) is specified. Members with default access are accessible only within the same package, meaning they cannot be accessed from [classes](#) in other packages. This helps maintain a level of encapsulation within a logical grouping of related classes.

Example

```
class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = 1000.0;
        account.deposit(500.0);
        System.out.println("Balance: " + account.balance); // Accessible because Main is in the same package
    }
}
// File: BankAccount.java
class BankAccount {
    double balance; // Default access modifier (package-private)
    void deposit(double amount) { // Default access modifier (package-private)
        balance += amount;
    }
}
Try it Yourself >>
```

Explanation

The balance variable and deposit method of BankAccount in this example have default (package-private) access, meaning that Main, which is in the same package, can access them. The default access level is illustrated by the Main class, which initializes the balance, deposits money, and prints the updated balance.

Output

```
Balance: 1500.0
```

2. Private Access Modifier in Java

The private access modifier restricts access to class members within the same class only. When a variable or method is declared private, it cannot be accessed from outside its own class. This modifier is essential for

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

data hiding and encapsulation, helping to protect internal implementation details and maintain control over how data is accessed or modified.

Example

```
class BankAccount {
    private double balance;
    public BankAccount(double initialBalance) {
        balance = initialBalance;
    }
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println(amount + " deposited successfully.");
        } else {
            System.out.println("Invalid amount for deposit.");
        }
    }
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println(amount + " withdrawn successfully.");
        } else {
            System.out.println("Invalid amount or insufficient balance.");
        }
    }
    public void displayBalance() {
        System.out.println("Current balance: " + balance);
    }
    public static void main(String[] args) {
        // Example usage
        BankAccount account = new BankAccount(1000);
        account.displayBalance();
        account.deposit(500);
        account.displayBalance();
        account.withdraw(200);
        account.displayBalance();
        account.withdraw(1500);
    }
}
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

```
}  
}  
Try it Yourself >>
```

Explanation

The BankAccount class employs private access for balance to enable secure data management. Deposit and withdraw methods validate the balance, while displayBalance displays the current amount. The main method demonstrates these actions by showing how encapsulation controls and protects data within the class.

Output

```
Current balance: 1000.0  
500.0 deposited successfully.  
Current balance: 1500.0  
200.0 withdrawn successfully.  
Current balance: 1300.0  
Invalid amount or insufficient balance.
```

3. Protected Access Modifiers in Java

The protected access modifier allows access to class members within the same package and also to subclasses in other packages. It offers a level of access broader than private but more restricted than public. This modifier is commonly used when you want to allow inheritance-based access to certain members while still preventing access from unrelated external classes.

Example

```
// Base class  
class Vehicle {  
    protected String brand; // protected member variable  
    protected void displayDetails() {  
        System.out.println("Brand: " + brand);  
    }  
}  
  
// Derived class  
class Car extends Vehicle {  
    private int mileage; // private member variable specific to Car  
    public Car(String brand, int mileage) {  
        this.brand = brand; // accessing protected member from superclass  
    }  
}
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

```
        this.mileage = mileage;
    }
    public void displayCarDetails() {
        displayDetails(); // accessing protected method from superclass
        System.out.println("Mileage: " + mileage);
    }
}
// Main class to test inheritance and protected access
class Main {
    public static void main(String[] args) {
        Car myCar = new Car("Toyota", 30);
        myCar.displayCarDetails();
    }
}
Try it Yourself >>
```

Explanation

There are two classes in this Java example: Parent and Child. The notion of inheritance and protected access is shown by the reality that the Child class derives from the Parent class and can access the protected method "display()" of the Parent class thanks to the protected access modifier.

Output

```
Brand: Toyota
Mileage: 30
```

4. Public Access Modifier in Java

The public access modifier allows classes, methods, and variables to be accessed from anywhere in the program, regardless of the package they belong to. When a member is declared public, it becomes universally accessible across all classes and packages. This modifier is typically used when you want to expose functionality or data for widespread use throughout the application.

Example

```
// File: Main.java
class Main {
    public static void main(String[] args) {
        Book book = new Book();
        book.title = "The Great Gatsby";
    }
}
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

```
        book.setAuthor("F. Scott Fitzgerald");

        System.out.println("Title: " + book.title);
        System.out.println("Author: " + book.getAuthor());
    }
}
// File: Book.java
class Book {
    public String title; // Public access modifier

    private String author; // Private access modifier

    public void setAuthor(String author) { // Public access modifier
        this.author = author;
    }
    public String getAuthor() { // Public access modifier
        return this.author;
    }
}
```

Run Code >>

Explanation

This code defines the Main and Book classes. Main creates a new instance of Book, using public methods (setAuthor) to set its title and author and public access (title and getAuthor) to obtain it. A book that allows for private access to control direct modification encapsulates the author field.

Output

```
Title: The Great Gatsby
Author: F. Scott Fitzgerald
```

Advantages of Access Modifiers in Java

- Encapsulation & Security – Protect sensitive data by hiding it from outside access.
- Better Code Structure – Helps organize code into well-defined modules.
- Improved Maintainability – Limits what can be accessed or modified, reducing bugs.
- Controlled Inheritance – Allows selective visibility across packages and subclasses.
- API Safety – Exposes only necessary methods, keeping internal logic hidden.
- Avoids Accidental Changes – Prevents misuse of internal class members.

Disadvantages of Access Modifiers in Java

- Overuse Can Limit Flexibility – Making too many members private or protected can restrict reuse or extension.
- Can Cause Tight Coupling – Improper use may force developers to write tightly dependent code or use workarounds (like getters/setters).
- Increased Complexity – Beginners may find it confusing to decide the right modifier, especially in large projects.
- Harder Debugging Across Packages – Restricted access may make it difficult to trace or debug behavior between classes in different packages.
- Slows Down Development (Initially) – Overthinking access levels can delay coding in the early stages of a project.

12-Abstraction

Abstract Classes and Methods

Data **abstraction** is the process of hiding certain details and showing only essential information to the user. Abstraction can be achieved with either **abstract classes** or **interfaces** (which you will learn more about in the next chapter).

The **abstract** keyword is a non-access modifier, used for classes and methods:

- **Abstract class:** is a restricted class that cannot be used to create objects (to access it, it must be inherited from another class).
- **Abstract method:** can only be used in an abstract class, and it does not have a body. The body is provided by the subclass (inherited from).

An abstract class can have both abstract and regular methods:

```
abstract class Animal {  
  
    public abstract void animalSound();  
  
    public void sleep() {  
  
        System.out.println("Zzz");  
  
    }  
  
}
```

From the example above, it is not possible to create an object of the Animal class:

```
Animal myObj = new Animal(); // will generate an error
```

To access the abstract class, it must be inherited from another class. Let's convert the Animal class we used in the Polymorphism chapter to an abstract class:

```
abstract class Animal {  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
    public abstract void animalSound();  
    public void sleep() {  
        System.out.println("Zzz");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Pig p = new Pig();  
        p.setName("Pig");  
        p.animalSound();  
        p.sleep();  
    }  
}
```

```
        System.out.println("Animal Name: " + p.getName());  
    }  
}
```

13-Encapsulation

Encapsulation is one of the fundamental principles of Object-Oriented Programming (OOP). It is the concept of wrapping data (variables) and methods that operate on the data into a single unit called a class. It helps in restricting direct access to certain details of an object and only exposing necessary information through methods.

Why Encapsulation?

- **Data Hiding:** It prevents direct access to class fields, ensuring better control over data.
- **Modularity:** A class with encapsulated fields and methods makes it easier to manage code.
- **Maintainability:** Changing the internal implementation does not affect external code.
- **Security:** Restricts unauthorized access to sensitive data.

How to Implement Encapsulation?

Encapsulation in Java is implemented by:

- Declaring class variables as private.
- Providing public getter and setter methods to access and update the private fields.

Example: Bank Account

Let's consider a bank account where we encapsulate the balance to ensure controlled access.

```
class BankAccount {  
  
    private double balance; // private variable  
  
    public BankAccount(double initialBalance) {  
  
        if (initialBalance > 0) {  
  
            this.balance = initialBalance;  
  
        }  
  
    }  
}
```

```
public double getBalance() {  
    return balance;  
}  
  
public void deposit(double amount) {  
    if (amount > 0) {  
        balance += amount;  
        System.out.println("Deposited: " + amount);  
    } else {  
        System.out.println("Invalid deposit amount.");  
    }  
}  
  
public void withdraw(double amount) {  
    if (amount > 0 && amount <= balance) {  
        balance -= amount;  
        System.out.println("Withdrawn: " + amount);  
    } else {  
        System.out.println("Insufficient funds or invalid amount.");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // Create a bank account object  
        BankAccount account = new BankAccount(5000);  
        System.out.println("Initial Balance: " + account.getBalance());  
    }  
}
```

```
        account.deposit(2000);

        System.out.println("Updated Balance: " + account.getBalance());

        account.withdraw(1000);

        System.out.println("Final Balance: " + account.getBalance());
    }
}
```

Expected Output

Initial Balance: 5000.0

Deposited: 2000.0

Updated Balance: 7000.0

Withdrawn: 1000.0

Final Balance: 6000.0

How Encapsulation Is Achieved in This Example

1. The balance field is declared private, so it can't be accessed directly from outside the class.
2. The getBalance(), deposit(), and withdraw() methods provide controlled access to the field.
3. Both deposit() and withdraw() include validation logic, ensuring that incorrect values cannot affect the internal state.

14-Scanner and System Class

In Java, Scanner and System are two fundamental, predefined classes used to manage core tasks like user input, console output, and system-level operations. While Scanner is primarily a utility tool for reading and parsing data, System is a final, uninstantiable class providing access to the underlying platform and runtime environment. [[1](#), [2](#), [3](#), [4](#)]

The Scanner Class

The Scanner class belongs to the java.util package. It is designed to break text or stream inputs into tokens (pieces) using a delimiter (whitespace by default) and parse them into specific data types. [[1](#), [2](#), [3](#)]

Core Methods for Input

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

- `next()`: Scans and returns the next complete token (single word) as a string.
- `nextLine()`: Reads and returns an entire line of text including spaces.
- `nextInt()` / `nextDouble()`: Parses the next token as an integer or decimal value.
- `hasNextInt()` / `hasNextDouble()`: Checks if the next token can be successfully parsed into that specific type.
- `close()`: Closes the scanner to release underlying system resources. [[1](#), [2](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#)]

The System Class

The System class belongs to the `java.lang` package and is imported automatically. You cannot create an object of this class; all of its variables and methods are static. [[1](#), [2](#), [3](#)]

1. Standard Streams (Fields)

- `System.in`: The standard input stream, configured by default to receive data from the keyboard.
- `System.out`: The standard output stream, typically used via `System.out.println()` to display data on the console.
- `System.err`: The standard error stream, used to output error logs (often formatted in red in modern IDEs). [[1](#), [2](#), [3](#), [4](#), [5](#)]

2. Frequently Used Methods

- `System.currentTimeMillis()`: Returns the current time in milliseconds since January 1, 1970 (useful for tracking execution speed).
- `System.arraycopy()`: A highly efficient, native method used to copy data from one array to another.
- `System.exit(int status)`: Terminates the currently running Java Virtual Machine (JVM). A status of 0 indicates successful termination.
- `System.getProperty(String key)`: Retrieves specific platform information, such as `user.name` or `os.name`. [[1](#), [2](#), [3](#), [4](#), [5](#)]

Feature	Scanner Class	System Class
Package	<code>java.util</code> (must be explicitly imported)	<code>java.lang</code> (implicitly imported)
Instantiation	Requires creating an instance	Cannot be instantiated; used directly

	using new Scanner(...)	via class name
Primary Focus	Reading and parsing string or primitive data types	Interfacing with the JVM, OS, and standard streams
Input Sources	Keyboard (System.in), text files, or strings	System hardware, configuration parameters, environment variables

This code utilizes both classes to read keyboard data via System.in, calculate code execution times, and print output via System.out: [[1](#), [2](#), [3](#)]

```
import java.util.Scanner; // Required for Scanner class

public class Main {

    public static void main(String[] args) {

        // Record the start time using the System class

        long startTime = System.currentTimeMillis();

        // Instantiate the Scanner by passing the System standard input stream

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name: ");

        String name = scanner.nextLine(); // Reads the whole line

        System.out.print("Enter your age: ");

        int age = scanner.nextInt(); // Reads an integer

        // Output to the terminal using the System standard output stream

        System.out.println("Hello " + name + "! You are " + age + " years old.");

        // Always close your scanner to prevent resource leaks

        scanner.close();

        // Calculate and print program duration
```

```
        long endTime = System.currentTimeMillis();

        System.out.println("Execution time: " + (endTime - startTime) + " ms");

    }

}
```

Polymorphism

What is Polymorphism?

Polymorphism is derived from two Greek words, “poly” and “morph”, which mean “many” and “forms”, respectively. Hence, polymorphism meaning in Java refers to the ability of objects to take on many forms. In other words, it allows different objects to respond to the same message or method call in multiple ways.

Polymorphism allows coders to write code that can work with objects of multiple classes in a generic way without knowing the specific class of each object.

Types of Polymorphism

1. **Compile-time Polymorphism (Method Overloading)**: Achieved by defining multiple methods with the same name but different parameters within the same class.
2. **Runtime Polymorphism (Method Overriding)**: Achieved when a subclass provides a specific implementation of a method already defined in its superclass.

Java Abstract Method

An **abstract method** is a method that is **declared without a body (implementation)**. It is declared using the **abstract** keyword inside an **abstract class**. The subclass is **required to provide the implementation** of the abstract method.

Syntax

```
abstract returnType methodName();
```

Implementing Abstract Methods

If the abstract class includes any abstract method, then all the child classes inherited from the abstract superclass must provide the implementation of the abstract method. For example,

```
// Abstract class
```

```
abstract class Language {
```

```
// Abstract method

abstract void display();

}

// Subclass

class Main extends Language {

    // Implementing the abstract method

    void display() {

        System.out.println("This is Java Programming");

    }

    public static void main(String[] args) {

        Main obj = new Main();

        obj.display();

    }

}
```

In the above example, we have created an abstract class Animal. The class contains an abstract method makeSound() and a non-abstract method eat().

We have inherited a subclass Dog from the superclass Animal. Here, the subclass Dog provides the implementation for the abstract method makeSound().

We then used the object d1 of the Dog class to call methods makeSound() and eat().

Note: If the Dog class doesn't provide the implementation of the abstract method makeSound(), Dog should also be declared as abstract. This is because the subclass Dog inherits makeSound() from Animal.

Java Abstract Class

An **abstract class** in Java is a restricted class declared with the `abstract` keyword that **cannot be instantiated** directly to create objects. It serves as a blueprint for other classes, allowing you to

define a common structure, shared data fields, and partial implementation while forcing subclasses to provide specific behaviors.

Core Characteristics & Rules

- **No Instantiation:** You cannot use the `new` keyword to create an instance of an abstract class.
- **Mixed Methods:** It can contain both abstract methods (without a body) and concrete methods (with a body).
- **Abstract Enforcement:** If a class contains at least one abstract method, the class itself **must** be declared abstract.
- **Subclass Contract:** Any concrete (non-abstract) subclass extending an abstract class must override and implement all of its inherited abstract methods.
- **Constructors Allowed:** Even though it cannot be instantiated on its own, it can have constructors. These run automatically when a subclass object is created.
- **Instance Variables:** Unlike interfaces, an abstract class can possess instance variables (non-static and non-final fields) to hold state.

Code Example: Abstract Class & Implementation

The following example defines an abstract concept (`Animal`) and shows how specific subtypes implement its missing logic:

```
java
```

```
// Abstract class
```

```
abstract class Animal {
```

```
    String name;
```

```
// Constructor
```

```
    public Animal(String name) {
```

```
        this.name = name;
```

```
    }
```

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

// Abstract method (No body, must be implemented by subclasses)

```
public abstract void makeNoise();
```

// Concrete method (Has a body, inherited by subclasses)

```
public void sleep() {  
    System.out.println(name + " is sleeping... Zzz");  
}  
}
```

// Concrete subclass

```
class Dog extends Animal {  
    public Dog(String name) {  
        super(name); // Invokes the parent class constructor  
    }  
}
```

// Implementing the abstract method

```
@Override  
public void makeNoise() {  
    System.out.println(name + " says: Woof! Woof!");  
}  
}
```

// Execution Class

```
public class Main {  
    public static void main(String[] args) {  
        // Animal animal = new Animal("Generic"); // COMPILE ERROR: Cannot instantiate  
        // Create a subclass object using an abstract reference type  
        Animal myDog = new Dog("Buddy");  
        myDog.makeNoise(); // Outputs: Buddy says: Woof! Woof!  
    }  
}
```

```
        myDog.sleep();    // Outputs: Buddy is sleeping... Zzz
    }
}
```

Java Packages

A package in Java is used to group related classes. Think of it as **a folder in a file directory**. We use packages to avoid name conflicts, and to write a better maintainable code. Packages are divided into two categories:

- Built-in Packages (packages from the Java API)
- User-defined Packages (create your own packages)

Built-in Packages

The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.

The library contains components for managing input, database programming, and much much more.

The library is divided into **packages** and **classes**. Meaning you can either import a single class (along with its methods and attributes), or a whole package that contain all the classes that belong to the specified package.

To use a class or a package from the library, you need to use the import keyword:

Syntax:

```
import package.name.Class; // Import a single class
```

```
import package.name.*; // Import the whole package
```

Import a Class

If you find a class you want to use, for example, the Scanner class, **which is used to get user input**, write the following code:

Example

```
import java.util.Scanner;
```

In the example above, java.util is a package, while Scanner is a class of the java.util package.

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

To use the Scanner class, create an object of the class and use any of the available methods found in the Scanner class documentation. In our example, we will use the `nextLine()` method, which is used to read a complete line:

Example

Using the Scanner class to get user input:

```
import java.util.Scanner;

class Main {

    public static void main(String[] args) {

        Scanner myObj = new Scanner(System.in);

        System.out.println("Enter username");

        String userName = myObj.nextLine();

        System.out.println("Username is: " + userName);

    }

}
```

Import a Package

There are many packages to choose from. In the previous example, we used the Scanner class from the `java.util` package. This package also contains date and time facilities, random-number generator and other utility classes.

To import a whole package, end the sentence with an asterisk sign (*). The following example will import ALL the classes in the `java.util` package:

Example

```
import java.util.*;
```

User-defined Packages

To create your own package, you need to understand that Java uses a file system directory to store them. Just like folders on your computer:

Example

└─ root

└─ mypack

└─ MyPackageClass.java

To create a package, use the package keyword:

MyPackageClass.java

```
package mypack;
```

```
class MyPackageClass {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("This is my package!");
```

```
    }
```

```
}
```

Save the file as **MyPackageClass.java**, and compile it:

```
C:\Users\Your Name>javac MyPackageClass.java
```

Then compile the package:

```
C:\Users\Your Name>javac -d . MyPackageClass.java
```

This forces the compiler to create the "mypack" package.

The -d keyword specifies the destination for where to save the class file. You can use any directory name, like c:/user (windows), or, if you want to keep the package within the same directory, you can use the dot sign ".", like in the example above.

Question Bank

***Include GATE & Competitive Examination Questions also from the COs**

Part A – 1 Mark Questions (20) //Add Rows

Part A – 1 Mark Questions (20)

S.No	Question	Level
1	Which of the following is NOT an OOP feature? A) Encapsulation B) Inheritance C) Top-down approach	

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	D) Polymorphism Answer: C	
2	Bytecode is executed by A) JDK B) JRE C) JVM D) Compiler Answer: C	
3	JDK consists of A) JVM only B) JRE only C) JRE + development tools D) Compiler only Answer: C	
4	Objects created using new are stored in A) Stack B) Heap C) Method Area D) PC Register Answer: B	
5	Default access modifier is A) public B) private C) protected D) package(default) Answer: D	
6	Constructor name must A) be same as class B) be main C) be any name D) return int Answer: A	
7	static members belong to A) Object B) Class C) Package D) JVM Answer: B	
8	Scanner class belongs to A) java.io B) java.lang C) java.util D) java.net Answer: C	
9	Which method reads an integer? A) next() B) nextLine() C) nextInt() D) readInt() Answer: C	

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

10	System.out is A) Class B) Object C) Package D) Method Answer: B	
11	Java is platform independent because of A) OS B) JVM C) BIOS D) RAM Answer: B	
12	Memory for local variables A) Heap B) Stack C) ROM D) Cache Answer: B	
13	Which modifier gives widest access? A) private B) protected C) default D) public Answer: D	
14	Which is true? A) Objects are blueprint B) Class is instance C) Object is instance of class D) None Answer: C	
15	Abstraction hides A) Data B) Implementation C) Objects D) Methods Answer: B	
16	Encapsulation combines A) Data and methods B) Packages C) JVM and JDK D) Heap and Stack Answer: A	
17	Entry point is A) run() B) start() C) main() D) init() Answer: C	
18	Non-static method requires A) Class name	

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	B) Object C) Package D) JVM Answer: B	
19	Which is not in JRE? A) JVM B) Libraries C) Compiler D) Runtime Answer: C	
20	Output of static int x=5; print(x); A)0 B)5 C)Error D)Runtime Error Answer: B	

Part B – 2 Mark Questions (20)

S.No	Question	Level
1	Differentiate POP and OOP.	L2
2	Differentiate JVM, JRE and JDK.	L2
3	Explain Java platform independence.	L2
4	Differentiate class and object with an example.	L2
5	Compare static and non-static members.	L2
6	Why do constructors not have a return type?	L3
7	Differentiate default and parameterized constructors.	L2
8	Explain the need for access modifiers.	L2
9	Differentiate abstraction and encapsulation.	L2
10	Explain the purpose of the Scanner class.	L2
11	Explain the use of the System class.	L2
12	Differentiate heap and stack memory.	L3
13	Explain destructor does not supported by Java along with the concept of Garbage Collection and also explain how memory is automatically managed in Java.	L2
14	A single integer n = 314568 is given as input.Demonstrate the logic involved in reversing a number using loops using a Java program.	L3
15	Given an integer n followed by n integers {23, 46, 87, 75, 54, 46, 24, 32, 16},Illustrate sorting techniques to find the second largest element using a Java program.	L3
16	Employee name and basic salary are given as input. Interpret the concept of inheritance in Java to calculate the total salary by adding HRA and DA to the basic salary along the role of superclass and subclass.	L2
17	A number is called a Harshad Number if it is divisible by the sum of its digits.Extend a Java program to check whether a given number is a Harshad Number or not.	L1
18	Given a student's name and roll number as input,infer the role of access modifiers in Java in protecting data members and enabling access through	L2

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	member functions.Support your answer with a Java program.	
19	Two strings are given as input.S1="hello",S2="Hello world".Show the purpose of the equals() method in the Object class and demonstrate how it is used to compare two strings in Java..	L2
20	The number of objects to be created is given as input.Classify variables in Java and explain how a static variable is used to count the number of objects created, with the help of a Java program.	L2

//Add Rows

Part C – 14 Mark Questions (15)

S.No	Question	Level
1	Library Book Management Problem Statement Create a Book class with Book ID, Book Name, Author and Price. Initialize using a constructor and display books whose price is greater than Rs.500. Constraints Price > 0 Input Format Read input values as specified in the problem statement. Sample Input 3 101 JavaProgramming HerbertSchildt650 102 Python Basics	L6
2	Student Record Manager Problem Statement A college wants to store student information. Create a Java class named Student containing Roll Number, Name, Department and CGPA. Use a parameterized constructor to initialize the values. Create three student objects and display their details. Constraints - Roll Number > 0 - CGPA between 0.0 and 10.0 Input Format	L6

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	Read input values as specified in the problem statement. Sample Input 101 Revathy AI&DS 9.1 102 Karthik CSE 8.8 103 Priya IT 9.4 Sample Output Student Details Roll No : 101 Name : Revathy Department : AI&DS CGPA : 9.1 Roll No : 102 Name : Karthik Department : CSE CGPA : 8.8 Roll No : 103 Name : Priya Department : IT CGPA : 9.4	
3	Explain Java compilation and execution.	L2
4	Discuss classes and objects with program.	L2
5	Bank Account Manager Problem Statement Design a BankAccount class using encapsulation with private data members Account Number, Account Holder Name and Balance. Implement deposit(), withdraw() and display() methods. Constraints • Withdrawal <= Balance • Deposit > 0	L3

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	Input Format Read input values as specified in the problem statement. Sample Input 10001 Revathy 25000 Deposit 5000 Withdraw 10000 Sample Output Current Balance : 20000	
6	Apply Java constructors to design a program that demonstrates the use of default, parameterized, and copy constructors for initializing student details and displaying the initialized values.	L3
7	Implement Java access modifiers by creating multiple classes that demonstrate the visibility of private, default, protected, and public members within the same package and across different packages.	L3
8	Develop a Java application using abstraction and encapsulation to model a banking system where account details are hidden using encapsulation and transactions are performed through abstract methods implemented by derived classes.	L3
9	Build a Java program using the Scanner class to accept two numbers and a user's choice, then perform arithmetic operations (addition, subtraction, multiplication, division, and modulus) using conditional statements.	L4
10	Apply the methods of the Java System class to develop a program that demonstrates the use of currentTimeMillis(), nanoTime(), gc(), exit(), and arraycopy() with appropriate examples and outputs.	L3
11	Illustrate the Java memory layout by developing a program that creates multiple objects, local variables, static variables, and string literals, then explain how they are stored in Stack Memory, Heap Memory, Method Area, and String Constant Pool during program execution.	L2
12	i) Apply Java arrays, loops, and conditional statements to develop a program that accepts n integers as input and prints the longest contiguous increasing subsequence in the array. ii) Build a Java program using string manipulation techniques to compress a given string by replacing consecutive repeating characters with the character followed by its count (e.g., aaabbc → a3b2c1).	L3
13	A) Construct a Java program using methods and control statements to validate a password based on the following conditions: i. Minimum length of 8 ii. At least one uppercase letter iii. At least one digit	L3

Internal Quality Assurance Cell (IQAC)

KPR Institute of Engineering and Technology

	B)Develop a Java class using static members and constructors to track the highest value entered among all objects created, where each object stores an integer input.	
14	i)Apply single inheritance and method overriding to design a Java program that calculates electricity bill amount for domestic and commercial consumers using different tariff rules. ii)Build a Java program using an abstract class that defines a template for calculateSalary() and implement it in subclasses to compute salary for Permanent and Contract employees.	L3
15	Develop a Java application using encapsulation and abstraction to model an Employee Payroll System, where employee details are encapsulated using private data members and salary calculation is performed using abstract methods implemented by different employee types.	L3